

ProjectName – C/Java/Python

Rapport final

AIT-ISSAD MOHAMED SAID, BILAL AL FAYOUMI, AMARA KOUSSAY, GUILLEM CAUSS

9 avril 2026

Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Pellentesque iaculis molestie euismod. In hac habitasse platea dictumst. Duis eu gravida sem. Vivamus pulvinar, tellus auctor sollicitudin iaculis, quam massa blandit ligula, in mollis est tellus et velit. Nulla gravida sapien nibh, nec faucibus lorem aliquam sed. Curabitur rutrum elit purus, et congue neque venenatis vitae. Sed non magna nisi. Pellentesque euismod non ipsum a hendrerit. Pellentesque vehicula tincidunt lacinia. Sed rhoncus porttitor facilisis.

Etiam sed molestie odio. Nunc cursus sollicitudin est vitae posuere. Nullam pellentesque metus eu dui porttitor bibendum. Morbi egestas arcu ut urna dictum placerat non non nisl. Quisque non ante in eros cursus pulvinar. Sed augue libero, eleifend a urna eget, pellentesque condimentum odio. Sed sed dui non diam mattis interdum eget sed urna. Integer consectetur, erat id rutrum posuere, dui velit dictum mi, quis tincidunt magna diam sit amet nunc. Etiam dui augue, sollicitudin rhoncus nisl a, rhoncus tempor purus.

Suspendisse cursus erat et metus fermentum ullamcorper. Etiam eros arcu, dapibus quis efficitur ut, congue id nunc. Pellentesque aliquam venenatis convallis. Nulla nisi justo, aliquet quis lacus eget, aliquet maximus velit. Maecenas ex turpis, iaculis eget dignissim non, efficitur vel mi. Maecenas pharetra elit ornare sem suscipit ornare id id sapien. Vivamus sem neque, consectetur vitae aliquet non, finibus eu lectus. Fusce aliquam ipsum ut viverra vehicula. Donec ultrices varius nulla vel finibus. Donec convallis fringilla ante id porta.

1 Modélisation du jeu

1.1 Représentation de l'état du jeu

L'état d'une partie est centralisé dans `GameState`. La classe regroupe les positions des joueurs, les murs restants et posés, le graphe de connectivité et le joueur courant. Cette séparation entre état dynamique, contraintes de déplacement et progression du tour évite de disperser la logique métier.

Les positions utilisent un encodage linéaire :

$$node = row \times size + col.$$

Ce format simplifie l'accès aux structures indexées et les parcours. Les champs utilisés en exécution sont `board_size`, `current_player`, `player_positions`, `remaining_walls`, `vertical_walls`, `horizontal_walls` et `graph`. Cette représentation sert directement la validation des coups, l'historisation et la persistance.

1.2 Graphe du plateau et structures associées

Le plateau est modélisé par un graphe non orienté $G = (V, E)$: les sommets représentent les cases et les arêtes les déplacements élémentaires autorisés, conformément à `[?, ?]`. Le graphe est stocké en liste d'adjacence `dict[int, list[int]]`, adaptée à une grille creuse (degré maximal 4) avec un coût mémoire en $O(V + E)$.

Les murs sont appliqués comme suppressions d'arêtes plutôt que comme transformations géométriques `[?]`. La vérification de connectivité repose sur BFS, avec une complexité $O(V + E)$ sur graphe non pondéré `[?]`. Les distances de plus court chemin sont aussi réutilisées par les heuristiques IA, notamment pour Minimax et MCTS `[?, ?]`.

En complément, le moteur maintient `player_positions`, deux collections de murs (vertical/horizontal) et un historique de snapshots pour *undo/redo*.

1.3 Déplacements des pions et validation des murs

Les déplacements de base sont obtenus à partir des voisins du sommet courant dans le graphe. Lorsqu'un pion adverse occupe un sommet adjacent, la règle de saut s'applique si la case située derrière reste accessible ; sinon, des déplacements diagonaux sont autorisés selon les arêtes effectivement présentes. Ces cas dépendent de l'état courant des pions et sont traités dynamiquement dans `pawn_rules.py`.

La validation d'un mur combine deux contrôles : validité locale de placement (position, orientation, absence de chevauchement) et maintien d'un chemin vers la zone cible pour chaque joueur. L'implémentation effectue une suppression temporaire des arêtes concernées, exécute un BFS par joueur, puis valide la pose ou restaure l'état précédent. Sur une grille 9×9 , ce coût reste faible.

Le modèle retenu établit un lien direct entre structures de données et règles d'exécution : le graphe représente la connectivité, les règles dynamiques gèrent les cas contextuels et les vérifications globales assurent la validité de la partie.

2 Architecture logicielle

2.1 Architecture générale

Les Figures 1 et 2 présentent les composants principaux du projet en deux vues lisibles.

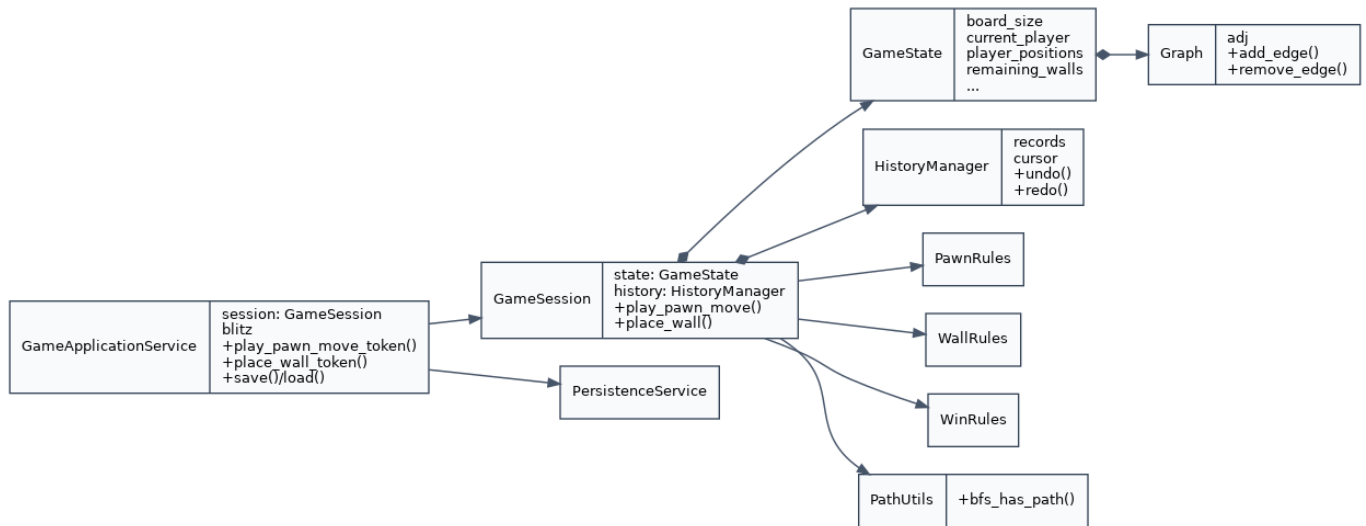


FIGURE 1 – Vue UML du noyau métier et de la couche applicative

L'architecture est organisée en couches : les modules d'interface et de réseau passent par la couche applicative, qui orchestre les opérations métier. **GameState** porte l'état, **GameSession** applique les transitions et **GameApplicationService** constitue le point d'entrée commun pour CLI, GUI, IA, persistance et réseau.

2.2 Modules et dépendances

Le module **core** contient le modèle métier (état, plateau, notation, snapshots). Le module **rules** implémente les validations de déplacements, de murs et de victoire. Le module **application** orchestre les cas d'usage et le cycle de partie. Les modules **interfaces** et **network** gèrent l'entrée/sortie utilisateur et les échanges client/serveur. Le module **utils** regroupe les composants techniques réutilisables (graphe, utilitaires), et **m1** porte les composants d'aide aux moteurs de recherche.

Les dépendances restent dirigées : **interfaces** → **application** → (**core**, **rules**, **utils**), avec **network** connecté à **application**. Cette orientation limite les cycles et isole le coeur métier.

```
quoridor/
|-- core/
|-- rules/
|-- application/
|-- interfaces/
|-- network/
'-- m1/
```

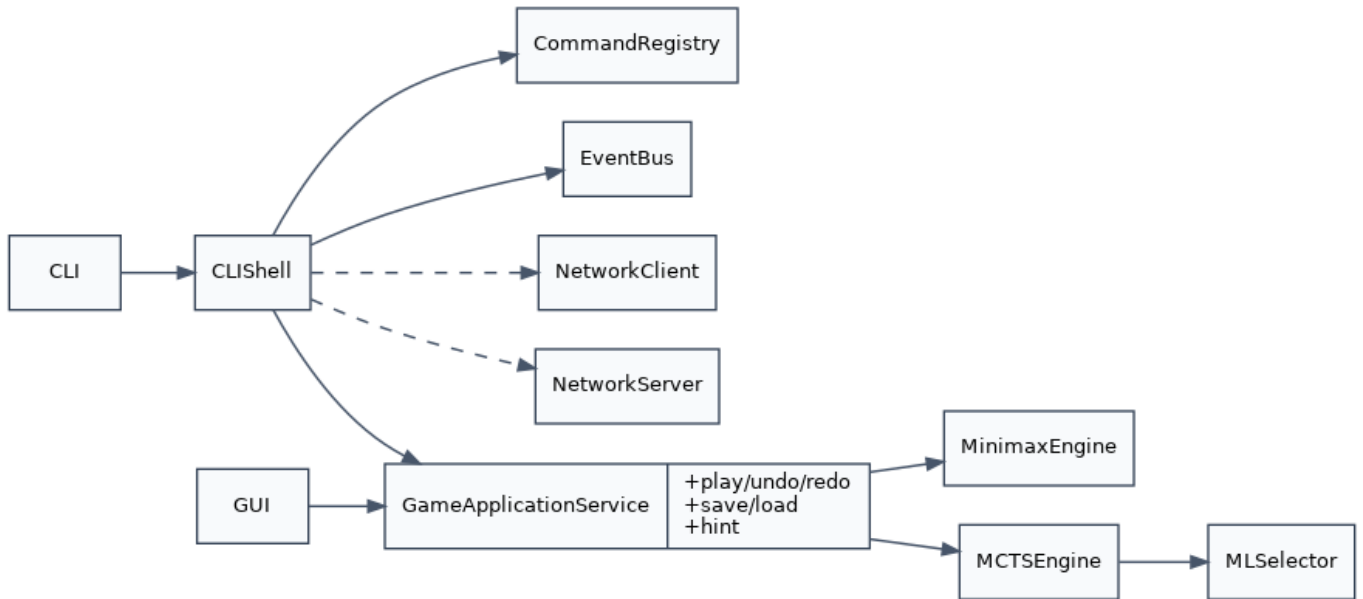


FIGURE 2 – Vue UML des interfaces, du réseau et des composants IA

2.3 Cheminement des actions

Toutes les actions passent par le même pipeline applicatif. Pour un déplacement utilisateur : interface → **GameApplicationService** → validation **rules** → mise à jour **GameSession/GameState** → historisation via **HistoryManager** → retour interface.

La pose de mur suit la même chaîne, avec simulation sur graphe et test BFS avant validation. Un coup IA suit également ce pipeline : **GameApplicationService** déclenche Minimax/MCTS, puis applique le coup sélectionné via les mêmes étapes de validation et de mise à jour.

2.4 Intégration des algorithmes

BFS est implémenté dans **rules/utils** pour les tests de connectivité. Les moteurs Minimax, alpha-bêta et MCTS sont orchestrés par **application**. Le mécanisme *undo/redo* repose sur **HistoryManager** et les snapshots d'état. La persistance et le parsing assurent la conversion entre formats externes et représentation interne, sans dupliquer la logique métier.

2.5 Design patterns utilisés

Command Dans le shell CLI, les commandes et **CommandRegistry** encapsulent les actions utilisateur et délèguent leur exécution à **GameApplicationService**. Cette structure découple saisie et exécution.

Strategy **PlayCommandStrategy**, **LocalPlayCommandStrategy** et **NetworkPlayCommandStrategy** isolent les variantes d'exécution local/réseau, sans multiplier les conditions dans la couche applicative.

Memento **GameSnapshot**, **MoveRecord** et **HistoryManager** conservent les états avant/après action, ce qui permet des restaurations cohérentes pour *undo/redo*.

Facade **GameApplicationService** joue le rôle de façade : il expose une API unique pour jouer un coup, poser un mur, gérer l'historique, piloter l'IA et charger/sauvegarder une partie.

Observer **EventBus** implémente un mécanisme d'observation léger entre producteurs et consommateurs d'événements, utile pour propager les mises à jour sans dépendances directes fortes.

L'ensemble produit une architecture compacte : responsabilités localisées, dépendances orientées vers le métier, et extensions (IA, réseau, interfaces) ajoutées sans modifier le noyau de règles.

3 Intelligence artificielle

3.1 Minimax avec élagage Alpha-Beta

Le Minimax explore récursivement l'arbre de jeu. À chaque nœud, il génère les coups légaux, applique chaque coup sur une copie de l'état, puis évalue les positions obtenues. Une victoire pour l'IA vaut +1 000 000, une défaite -1 000 000. À profondeur nulle, l'évaluation repose sur l'une des trois heuristiques configurables.

Sans élagage, le coût est exponentiel en fonction du facteur de branchement b et de la profondeur d : $O(b^d)$. Au Quoridor, ce facteur est élevé car il faut considérer à la fois les déplacements de pion et les poses de murs. L'élagage alpha-beta réduit ce coût en supprimant les branches qui ne peuvent plus influencer le résultat. La figure 3 montre ce cas simple : si MAX possède déjà une branche à 5, une branche adverse qui descend à 2 est coupée.

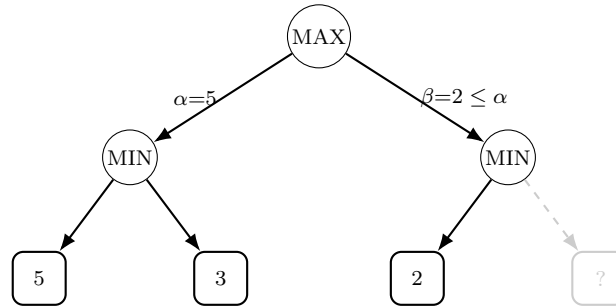


FIGURE 3 – Élagage alpha-beta : la branche droite est coupée car MIN a trouvé 2 alors que MAX dispose déjà d'une branche à 5.

Dans les modes à temps borné, la recherche vérifie une deadline avant chaque expansion de nœud. Si le temps est écoulé, elle s'arrête et renvoie le meilleur coup déjà calculé. Le pseudo-code correspondant est donné en figure 4.

```

fonction minimax(etat, profondeur, alpha, beta, maximise):
    verifier la deadline
    si etat terminal: retourner score final
    si profondeur = 0: retourner heuristique(etat)

    pour chaque coup legal:
        evaluer recursivement l'etat enfant
        mettre a jour la meilleure valeur courante
        mettre a jour alpha ou beta
        si beta <= alpha: arreter l'exploration

    retourner la meilleure valeur

```

FIGURE 4 – Pseudo-code du Minimax avec élagage alpha-beta

3.2 Iterative Deepening

Le problème d'un Minimax à profondeur fixe est simple : on ne sait pas à l'avance quelle profondeur sera atteignable dans le temps imparti. Au Quoridor, le coût varie fortement selon le nombre de murs encore disponibles.

L'Iterative Deepening lance donc plusieurs recherches successives à profondeur croissante : 1, puis 2, puis 3, etc. Quand la deadline est atteinte, on conserve le meilleur coup de la dernière profondeur complètement terminée. Ce choix garantit qu'un coup valide est toujours disponible, même sous une contrainte de temps stricte.

L'IA expose deux usages :

- **Profondeur explicite** (`-ai-minimax-depth`) : Minimax à profondeur fixe.
- **Temps limité** (`-ai-time`) : recherche itérative qui approfondit tant que le temps reste disponible.

Quand l'utilisateur choisit `minimax` sans préciser de profondeur, le programme convertit automatiquement le temps disponible en profondeur maximale selon les seuils du tableau ci-dessous.

Temps	Profondeur
≤ 1 s	1
≤ 3 s	2
≤ 6 s	3
≤ 12 s	4
> 12 s	5

3.3 Pipeline d'apprentissage automatique

Nous avons ajouté une politique de sélection ML pour le MCTS. L'objectif n'est pas de remplacer tout le moteur de recherche, mais uniquement la sélection du prochain enfant à explorer.

```

fonction iterative_deepening(etat, joueur_ia, temps_max, prof_max):
    fixer une deadline
    initialiser meilleur_coup avec un coup legal

    pour profondeur croissante:
        lancer minimax a cette profondeur
        si la profondeur est entierement terminee:
            memoriser le meilleur coup
        sinon:
            arreter

    retourner meilleur_coup

```

FIGURE 5 – Pseudo-code de l’Iterative Deepening

La figure 6 résume le pipeline complet : génération de parties en self-play, extraction des features, entraînement du modèle, puis intégration dans le MCTS.

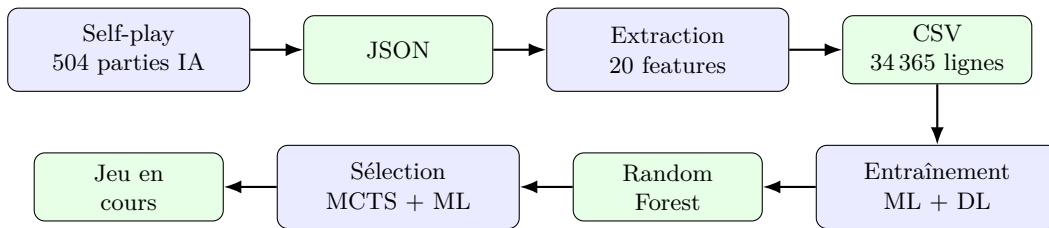


FIGURE 6 – Pipeline ML : génération, entraînement et intégration en jeu

Dataset. Le dataset est construit à partir de 504 parties jouées en self-play entre plusieurs variantes de nos IA : Minimax à profondeurs 1, 2 et 3 avec les trois heuristiques, et Iterative Deepening avec trois budgets de temps distincts. Chaque partie est rejouée tour par tour pour extraire un vecteur de 20 features par coup. On obtient 34 365 exemples au format tabulaire.

Les 20 features sont réparties en trois groupes : contexte avant le coup, description du coup et effet du coup. La figure 7 résume cette décomposition.

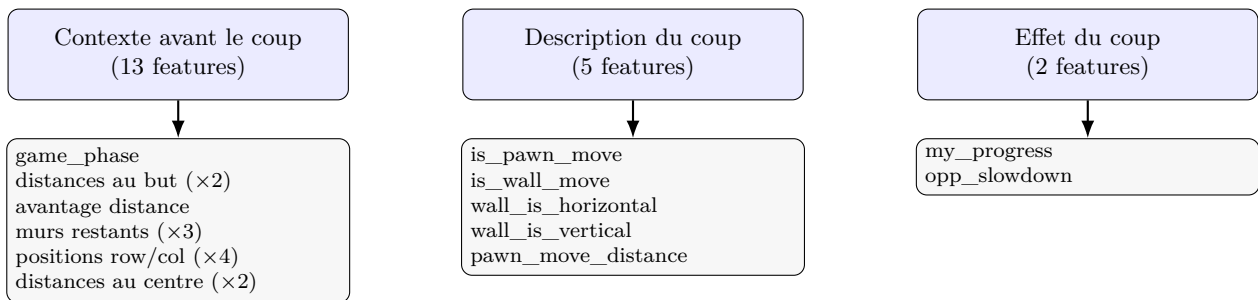


FIGURE 7 – Les 20 features utilisées pour la sélection ML

Entraînement. Les données sont séparées en 80 % apprentissage et 20 % test avec stratification. Nous avons comparé quatre modèles classiques de scikit-learn. Les résultats sont donnés dans le tableau 1.

Modèle	Train Acc	Test Acc	F1	AUC
Régression Logistique	0,757	0,755	0,759	0,833
Arbre de Décision ($d=5$)	0,840	0,836	0,842	0,931
Random Forest (100)	1,000	0,938	0,939	0,987
Gradient Boosting	0,869	0,866	0,867	0,946

TABLE 1 – Comparaison des modèles ML

Le Random Forest est le meilleur compromis sur nos données : AUC de 0,987, F1 de 0,939 et intégration simple

dans le moteur. Nous avons aussi testé un MLP Keras. Le MLP atteint une AUC de 0,975, inférieure au Random Forest. Nous avons donc retenu le Random Forest, plus précis et plus léger à intégrer.

Layer (type)	Output Shape	Param #
dense	(None, 128)	2,688
batch_normalization	(None, 128)	512
dropout	(None, 128)	0
dense_1	(None, 64)	8,256
batch_normalization_1	(None, 64)	256
dropout_1	(None, 64)	0
dense_2	(None, 32)	2,080
batch_normalization_2	(None, 32)	128
dropout_2	(None, 32)	0
dense_3	(None, 1)	33

Figure 8 – Architecture du MLP utilisé pour comparaison

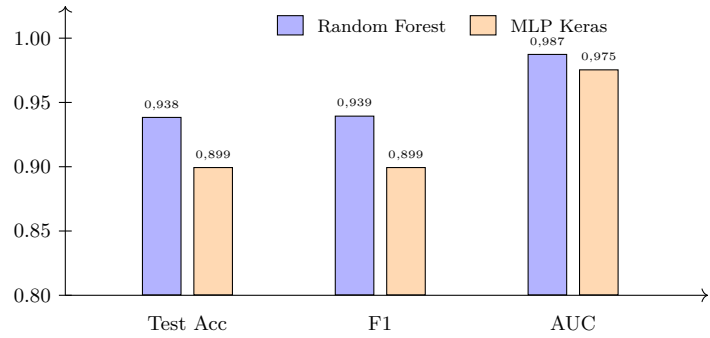


Figure 9 – Comparaison Random Forest / MLP sur le jeu de test

Intégration dans le MCTS. La sélection ML remplace uniquement la formule UCT. Pour chaque enfant d’un nœud, le programme reconstruit les 20 features à partir de l’état avant le coup, du coup lui-même et de l’état après le coup. Le modèle calcule ensuite une probabilité de victoire, et l’enfant retenu est celui dont cette probabilité est maximale. Les autres étapes du MCTS restent inchangées : expansion, rollout et rétro-propagation.

Le mode est activé via `-ai-mcts-selection ML`. Le modèle est chargé une seule fois puis mis en cache. Le pseudo-code est donné en figure 10. Il représente uniquement l’étape de sélection : pour chaque enfant déjà généré, on calcule un score ML, puis on choisit celui qui obtient la meilleure probabilité de victoire.

```
fonction ml_select_child(noeud_parent):
    pour chaque enfant du noeud:
        construire ses features
        estimer sa probabilite de victoire avec le modele

    retourner l'enfant avec le meilleur score
```

FIGURE 10 – Pseudo-code de la sélection ML dans le MCTS

Conclusion

Suspendisse sit amet nisi placerat, vestibulum nisl vel, iaculis nunc. Suspendisse quis libero vel ipsum gravida congue nec id arcu. Donec vel mi viverra, commodo urna vitae, tincidunt lectus. Fusce pellentesque, lectus a placerat vestibulum, magna lectus placerat nibh, vehicula pellentesque lacus erat non nulla. Suspendisse maximus, turpis at suscipit egestas, libero massa ultrices libero, at tincidunt velit eros a eros. Maecenas id tempor nunc. Phasellus condimentum porttitor sapien in sagittis. Proin in metus diam. Mauris in turpis sed ex viverra iaculis. Sed lobortis, tellus vitae congue blandit, leo elit auctor libero, id tempor ante dolor quis quam. Donec eget ligula quam. Maecenas porttitor massa vel felis elementum tristique.

A Liste des 40 besoins

Cette annexe synthétise l’état de conformité des besoins fonctionnels. Les verdicts utilisés sont limités à **FAIT**, **PARTIEL** et **NON FAIT**. Lorsque l’exigence n’a pas encore fait l’objet d’un audit stabilisé, l’évaluation reste prudente et devra être confirmée lors de la relecture finale.

Exigence	Intitulé court	Verdict	Justification concise	Preuve / référence
F1	Exigence F1 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F2	Exigence F2 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F3	Exigence F3 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F4	Exigence F4 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F5	Exigence F5 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider

Exigence	Intitulé court	Verdict	Justification concise	Preuve / référence
F6	Exigence F6 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F7	Annulation et rétablissement	PARTIEL	Le mécanisme existe, mais la conformité finale à la spécification doit être reconfirmée avant rendu.	Tests undo/redo à revalider
F8	Exigence F8 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F9	Exigence F9 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F10	Exigence F10 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F11	Exigence F11 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F12	Exigence F12 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F13	Exigence F13 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F14	Exigence F14 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F15	Shell interactif	PARTIEL	Le shell interactif est largement implémenté, mais la conformité globale n'a pas encore été consolidée exigence par exigence.	CLI et tests shell à revalider
F16	Exigence F16 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F17	Exigence F17 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F18	Exigence F18 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F19	Exigence F19 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F20	Déplacement et saut du pion	PARTIEL	Les règles de déplacement et de saut semblent présentes, mais la relecture finale de conformité reste nécessaire.	Règles du pion et tests à revalider
F21	Sauvegarde et restauration	FAIT	La sauvegarde et la restauration sont conformes, avec sections ordonnées, parseur robuste et messages d'erreur explicites.	Service de persistance ; tests save/load
F22	Commentaires dans la sauvegarde	FAIT	Les commentaires en ligne et en bloc sont correctement ignorés lors du parsing.	Parseur de sauvegarde ; tests CLI I/O
F23	Format des paramètres du jeu	FAIT	La section <code>[settings]</code> sérialise désormais les paramètres de partie et les principaux paramètres de programme au format clé-valeur.	Persistance ; tests program settings
F24	Format du plateau AS-CII	FAIT	Le format <code>[game]</code> est lu et écrit correctement, et les erreurs de lecture signalent maintenant le type d'erreur et la ligne concernée.	Contest parser ; tests contest
F25	Format de l'historique	FAIT	L'historique est écrit et relu sous forme de tours, puis rejoué pour reconstruire l'état de la partie.	Persistance ; tests history
F26	Module graphe générique	FAIT	Le projet fournit un module de graphe générique en liste d'adjacences, indépendant de Quoridor et réutilisable dans d'autres contextes.	<code>utils/graph.py</code> ; tests graphe
F27	Algorithmes sur les graphes	FAIT	Les algorithmes de validation, génération de coups, application, score et fin de partie sont bien intégrés au moteur.	Règles, session de jeu, tests
F28	Menus de la GUI	PARTIEL	Les menus et actions principales existent, mais la conformité textuelle à la spécification n'a pas encore été entièrement normalisée.	GUI ; tests menus à revalider

Exigence	Intitulé court	Verdict	Justification concise	Preuve / référence
F29	Exigence GUI à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F30	Glisser-déposer du pion	FAIT	Le déplacement du pion à la souris par glisser-déposer est implémenté et testé.	GUI drag and drop ; tests GUI
F31	Temps de réflexion borné	FAIT	L'option <code>-ai-time</code> borne la réflexion et l'IA joue le meilleur coup complet trouvé avant expiration.	Minimax itératif ; tests IA
F32	Minimax avec élagage alpha-bêta	FAIT	L'algorithme minimax avec <i>alpha-beta pruning</i> est implémenté et activé par défaut pour le mode IA.	Moteur minimax ; config et CLI
F33	Fonctions d'évaluation Minimax	FAIT	Trois fonctions d'évaluation sont disponibles et le scoring choisi est désormais propagé jusqu'au moteur minimax.	AI logic ; tests scoring
F34	Approfondissement itératif	FAIT	Le mode <code>iterative</code> réalise un approfondissement progressif de minimax en fonction du temps disponible.	Moteur itératif ; tests dédiés
F35	Profondeur de recherche Minimax	FAIT	La profondeur maximale peut être fixée explicitement ou déterminée automatiquement à partir du temps disponible.	CLI, session, tests profondeur
F36	Monte Carlo Tree Search	FAIT	Le mode <code>mcts</code> est implémenté et utilise UCT comme politique de sélection standard par défaut.	Moteur MCTS ; tests MCTS
F37	Sélection MCTS par apprentissage	FAIT	Une politique de sélection ML entraînée sur des parties jouées est disponible via <code>-ai-mcts-selection</code> ML.	Module ML, dataset, tests selector
F38	Exigence IA à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F39	Exigence IA à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider
F40	Exigence F40 à confirmer	PARTIEL	Évaluation à confirmer lors de la relecture finale ; audit détaillé non stabilisé à ce stade.	À revalider